

Tarea 1: Ingesta y Procesamiento de Datos Serverless

1. Creación de Tabla DynamoDB:

- Crea una tabla llamada proy-tabla con fecha como clave de partición.
- Define las columnas necesarias: fecha (para indexación), mes_ano (para diferenciación mensual), desviación y media.

2. Creación de Bucket S3:

- Crea un bucket S3 llamado proy-ingestadatos para subir el archivo CSV original.

3. Configuración de Notificaciones SNS:

- Crea un tema SNS estándar llamado proy-alarma-temperatura.
- Añade una suscripción de protocolo email con tu correo electrónico para recibir las alertas.

4. Función Lambda 1: Procesador de Datos (proy-funcion-procesador-datos):

- Crea la función con Python 3.9 y asigna el rol de ejecución LabRole.
- **Desencadenador:** Agrega el bucket S3 creado anteriormente para que la función se active al cargar el CSV.
- **Configuración:** Aumenta el tiempo de espera a 1 minuto para asegurar que la inserción en base de datos tenga tiempo suficiente.
- **Lógica:** La función debe obtener el archivo del bucket, leer el CSV, parsear la fecha para crear la variable mes_ano (formato mes-año) e insertar los ítems (fecha, mes_ano, media, desviacion) en la tabla DynamoDB .

5. Función Lambda 2: Notificación (proy-funcion-notificacion):

- Crea la función con configuración similar a la anterior.
- **Desencadenador:** Agrega la tabla DynamoDB (proy-tabla) con un tamaño del lote de 500.
- **Lógica:** Debe revisar los registros insertados (INSERT). Si la desviacion es mayor a 0.5, debe publicar un mensaje en el tema SNS creado (proy-alarma-temperatura) indicando la fecha y la desviación detectada.

6. Comprobación:

- Sube particiones del archivo CSV al bucket S3 y verifica en DynamoDB que los datos se han cargado (aprox. 380 elementos) y que has recibido correos de alerta (aprox. 36 correos).
-

Tarea 2: Infraestructura y Aplicación Web (Docker)

1. Despliegue de Infraestructura Básica (CloudFormation):

- Crea una pila llamada proy-pila utilizando la plantilla proy-plantilla.
- Esto desplegará automáticamente:
 - **Red:** Una VPC (proy-vpc), dos subredes públicas en zonas de disponibilidad distintas, tabla de enrutamiento y un Internet Gateway.
 - **Grupos de Seguridad:**
 - FlaskSecurityGroup: Puertos 5000 (web) y 22 (SSH) abiertos.
 - BalanceadorSecurityGroup: Puerto 80 abierto al público.
 - TaskSecurityGroup: Puerto 5000 abierto solo al tráfico del balanceador.
 - **Instancia EC2:** Una instancia DockerInstance (t2.micro) con el par de claves proy-keypair y el grupo de seguridad Flask asociado.

2. Desarrollo de la Aplicación Flask:

- Conéctate vía Putty a la DockerInstance y crea un directorio flask_docker.
- Crea tres archivos dentro:
 - **funcion.py:** La aplicación Flask. Debe conectar con DynamoDB (boto3), definir funciones auxiliares para calcular medias y definir las rutas /temp, /maxdiff, /sd y /health para consultar la base de datos. Debe ejecutarse en el puerto 5000.
 - **requirements.txt:** Lista de dependencias (Flask, boto3, etc.).
 - **Dockerfile:** Usa imagen base python:3.8-alpine, establece directorio de trabajo /app, copia archivos, instala dependencias y ejecuta funcion.py.

3. Construcción y Prueba de Docker:

- Construye la imagen: `sudo docker build -t flask_container ..`
- Ejecuta el contenedor mapeando el puerto 5000: `sudo docker run -p 5000:5000 -d flask_container.`

- Verifica el funcionamiento haciendo peticiones curl o desde el navegador.

4. Subida a Amazon ECR (Elastic Container Registry):

- Crea un repositorio privado llamado proy-repo.
 - Obtén el token de autenticación de ECR mediante AWS CLI.
 - Etiqueta la imagen local (docker tag) con la URI del repositorio.
 - Sube la imagen (docker push) al repositorio ECR.
-

Tarea 3: Orquestación de Contenedores (ECS)

1. Punto de Enlace VPC (VPC Endpoint):

- Crea un punto de enlace tipo *Gateway* para DynamoDB (com.amazonaws.us-east-1.dynamodb) dentro de la VPC proy-vpc y asócialo a las tablas de enrutamiento .

2. Balanceador de Carga (ALB):

- Crea un Balanceador de Carga de Aplicaciones llamado proy-alb, expuesto a internet.
- Selecciona la proy-vpc y las dos subredes públicas.
- Asigna el grupo de seguridad BalanceadorSecurityGroup.
- Configura el agente de escucha en HTTP puerto 80.
- **Grupo de Destino:** Crea un grupo nuevo llamado proy-tg-flask, tipo direcciones IP, protocolo HTTP puerto 5000. Configura la comprobación de estado (health check) en la ruta /health con umbral de buen estado en 2 e intervalo de 6.

3. Clúster ECS:

- Crea un clúster llamado proy-cluster usando infraestructura Fargate e instancias EC2.
- Configuración: Aprovisionamiento bajo demanda, tipo de instancia t2.micro, rol LabInstanceProfile. Capacidad mínima 2, máxima 4.
- Red: Selecciona la VPC y subredes creadas. Asigna el grupo de seguridad TaskSecurityGroup y **habilita** la asignación automática de IP pública.

4. Definición de Tarea (Task Definition):

- Crea proy-task con tipo de lanzamiento EC2 y modo de red awsvpc.
- Asigna recursos: 1 vCPU y .2 GB de memoria. Roles de tarea y ejecución: LabRole.

- **Contenedor:** Añade el contenedor proy-container. Usa la URI de la imagen subida a ECR. Mapea el puerto 5000 TCP. Establece límite estricto de memoria en .2 GB y flexible en .1 GB.

5. Creación del Servicio ECS:

- Dentro del clúster, crea el servicio proy-ecs-service con tipo de lanzamiento EC2.
- Tipo de servicio: RÉPLICA, con 2 tareas deseadas.
- Selecciona la VPC, subredes y el grupo de seguridad TaskSecurityGroup.
- **Equilibrio de carga:** Selecciona "Balanceador de carga de aplicaciones", elige el contenedor proy-container, el balanceador proy-alb y el grupo de destino proy-tg-flask.

6. Verificación Final:

- Comprueba que las pilas de CloudFormation están en estado CREATE_COMPLETE.
- Verifica que las tareas en el servicio están en ejecución (RUNNING).
- Confirma que los destinos en el grupo de destino están en estado Healthy.
- Accede a la aplicación a través del nombre DNS del balanceador de carga para probar las rutas (/temp, /maxdiff, etc.).